

UF0Catch ゲームプロジェクト

技術アピール資料

DirectX & C++ ゲーム開発

プロジェクト概要

ゲームコンセプト: UF0を操作して人間に見つからないように動物を吸い込むステルスアクションゲーム

使用技術:

- 開発言語: C++
- グラフィックスAPI: DirectX 11
- アーキテクチャ: コンポーネントベースのゲームエンジン
- デザインパターン: State Pattern, Object Pool, Singleton

開発形態: 個人開発

実装規模: 約10,000行以上の独自実装コード

4つの主要技術実装

1. 空間分割アルゴリズム（線形4分木 + モートン順序）

実装箇所: Src/09_Energy/Liner4Tree.h

実装内容:

- 4分木による効率的な衝突判定の最適化
- モートン曲線（Z曲線）を使ったビット演算による空間インデックス
- 衝突判定の計算量を $O(n^2)$ から $O(n \log n)$ に削減

技術的なポイント:

- ビット分離アルゴリズムによるモートン符号化
- 階層的なセル探索による空間クエリ
- テンプレートによる汎用的なゲームオブジェクトコンテナ

```
// Morton number calculation with bit separation
uint16_t BitSeparate(uint16_t n) {
    n = (n | (n << 8)) & 0x00ff00ff;
    n = (n | (n << 4)) & 0x0f0f0f0f;
    n = (n | (n << 2)) & 0x33333333;
    n = (n | (n << 1)) & 0x55555555;
    return n;
}

uint16_t Get2DMortonNumber(float worldX, float worldY) {
    uint16_t cellX = normalizeCoord(worldX);
    uint16_t cellY = normalizeCoord(worldY);
    return BitSeparate(cellX) | (BitSeparate(cellY) << 1);
}
```

2. 幾何学的衝突判定システム

実装箇所: Src/11_GameSystem/VisionSystem.h/cpp

実装内容:

- AI視界システムのための扇形と円の高度な衝突判定
- 2次方程式の判別式を使った線分と円の交差判定
- 敵AIの発見メカニクスへの応用

技術的なポイント:

- 内積とベクトル正規化を用いた数学的アプローチ
- 早期リジェクションテストによる計算の最小化
- 扇形の境界や円弧の交差などのエッジケースに対応

```
// Line segment-circle intersection detection
bool LineSegmentCircleIntersection(
    const VECTOR2& lineStart, const VECTOR2& lineEnd,
    const VECTOR2& circleCenter, float circleRadius) const {

    VECTOR2 direction = lineEnd - lineStart;
    VECTOR2 toCenter = lineStart - circleCenter;

    float a = dot(direction, direction);
    float b = 2.0f * dot(toCenter, direction);
    float c = dot(toCenter, toCenter) - radius * radius;

    float discriminant = b * b - 4.0f * a * c;
    if (discriminant < 0.0f) return false;

    float t1 = (-b - sqrt(discriminant)) / (2.0f * a);
    float t2 = (-b + sqrt(discriminant)) / (2.0f * a);

    return (t1 >= 0 && t1 <= 1) || (t2 >= 0 && t2 <= 1);
}
```

3. デザインパターンの実装 (State Pattern)

実装箇所: Src/12_Tutorial/TutorialState.h, Src/09_Energy/Human/State/

実装内容:

- 複雑な振る舞い管理のためのステートマシン
- チュートリアル進行: Move → Suction → Expand → Discovery → Play
- 敵AIの状態遷移: Idle → Walk → FindPlayer

技術的なポイント:

- 関心の分離: ActorクラスとStateクラスの分離
- `std::unordered_map` による $O(1)$ の状態検索
- 事前割り当てによる実行時のメモリ確保の排除
- 状態遷移時のフレームレート低下を防止

```
// State pattern implementation
class CTutorialState {
public:
    virtual void Enter() {}
    virtual void Update() {}
    virtual void Exit() {}

    enum class State {
        Move, Suction, Expands, Discovery, Play
    };
};

class CMoveState : public CTutorialState {
    void Update() override {
        // Handle movement tutorial logic
        if (playerMoved()) transitionToNext();
    }
};

// State management with unordered_map
std::unordered_map<StateType, std::unique_ptr<State>> states;
```

4. ゲームシステムアーキテクチャ設計

実装箇所: Src/07_Scene/, Src/08_Player/, Src/11_GameSystem/

実装内容:

- ライフサイクル制御を持つシーン管理システム
- ゲームエンティティのライフサイクルを管理するオブジェクトマネージャ
- ゲームメカニクス: 円錐形吸引、経験値システム、Lerp補間

技術的なポイント:

- DeltaTimeを使用したフレームレート非依存の物理演算
- 吸引範囲の円錐体積計算
- イージングカーブを使った高さベースの吸引速度
- 内分比による地面平面への投影計算

```
// Suction displacement calculation
VECTOR3 CalcSuctionDisplacement(float moveTime,
                                const VECTOR3& animalPos) const {
    // Calculate speed multiplier based on height
    float heightDiff = m_coneTopPos - animalPos.y;
    float progress = 1.0f - (heightDiff / m_coneTopPos);
    float eased = progress * progress * progress; // Cubic easing

    float speedMultiplier = lerp(minSpeed, maxSpeed, eased);

    // Project to ground plane using internal division
    float ratio = (0 - animalPos.y) / (animalPos.y - m_coneTopPos);
    VECTOR3 groundPoint = VECTOR3(
        animalPos.x + ratio * (animalPos.x - transform.position.x),
        0,
        animalPos.z + ratio * (animalPos.z - transform.position.z)
    );

    VECTOR3 pullVector = transform.position - groundPoint;
    return pullVector / moveTime * speedMultiplier * DeltaTime();
}
```

まとめと成果

コード品質:

- 「lizard」ツールを使用して循環的複雑度を測定
- 小さく焦点を絞った関数で低い複雑度スコアを維持
- コードの可読性、テスト容易性、保守性を向上

パフォーマンス最適化:

- 空間分割により衝突判定を約90%削減
- 状態の事前割り当てにより遷移時のフレーム落ちを排除
- フレームレート非依存の物理演算で一貫したゲームプレイを実現

ソフトウェアエンジニアリング原則:

- ステート設計における単一責任の原則
- テンプレートプログラミングによるDRY原則
- アーキテクチャ全体でSOLID原則を適用

習得技術:

これらの実装により以下の技術を実証:

- 高度なC++プログラミング (テンプレート、STL、モダンC++11/14機能)
- アルゴリズム設計と最適化
- グラフィックスプログラミング (DirectX)
- ソフトウェアアーキテクチャとデザインパターン
- パフォーマンスを意識した開発
- ゲームメカニクスのための数学的問題解決